LESSON 06 · CHEAT SHEET

Stream Fundamentals

Java 8 → Java 21 · results use the shared course dataset
course "today" = 2026-11-07

Key rules

1. **Pipeline = source → 0..n intermediate → 1 terminal.** 8
2. **Lazy:** nothing runs until the terminal op. No terminal means no work at all.
3. **Vertical order (fusion):** each element goes through the whole pipeline before the next one starts.
4. **Stateful barrier:** `sorted` (and `distinct`) must see the input first; `sorted` holds everything.
5. **Short-circuit:** `limit`, `takeWhile`, `anyMatch`, `findFirst`… stop early, so infinite sources are fine.
6. **One-shot:** a second terminal call throws `IllegalStateException`. Keep the list or a `Supplier<Stream>`.
7. **Stores nothing** and never modifies its source.
8. `parallel()`/`sequential()` set one flag; **the last call wins**.

Canonical snippets

1 · Top 3 debits (after/StreamTopDebits)

```
txns.stream()
    .filter(t -> t.type() == DEBIT)
    .sorted(Comparator.comparing(Transaction::amount)
                      .reversed())
    .limit(3)       // sort FIRST, then limit
    .toList();      // [16]
```

2 · Sources

```
Stream.iterate(1, n -> n * 2)          // [8] infinite
Stream.iterate(start, d -> !d.isAfter(end),
               d -> d.plusWeeks(1))  // [9]
Stream.ofNullable(x)          // [9] 0 or 1
IntStream.rangeClosed(10, 12) // 10 11 12
try (var l = Files.lines(csv)) {…} // close
```

3 · Reuse safely

```
Supplier<Stream<Payment>> s = list::stream;
s.get().count(); s.get().anyMatch(p);  // fresh
```

Method map (core ops)

Operation	Kind	Notes	Since
`filter · map · peek`	inter.	stateless	8
`flatMap · mapToInt`	inter.	stateless	8
`mapMulti`	inter.	stateless, 1→n	16
`distinct · sorted`	inter.	**stateful**	8
`limit · skip`	inter.	stateful; limit short-circuits	8
`takeWhile · dropWhile`	inter.	stateful; takeWhile short-circuits	9
`forEach · count · reduce`	term.	visits all	8
`collect · min · max`	term.	visits all	8
`anyMatch · findFirst`	term.	short-circuit	8
`toList()`	term.	unmodifiable	16
`gather(Gatherer)`	inter.	custom op	Beyond 21 · 24

Laziness trace (theory/LazinessAndFusion)

```
filter(amount ≥ 100) → map(id) → limit(2)
filter P001, map P001, filter P002,
filter P003, map P003        → [P001, P003]
```

With `sorted()` between them: all **filter** calls P001–P005 come first, then **map** P001, P003, P005.

Mini diagram: the conveyor belt



The terminal op pulls one item at a time through the belt.

Real results from the course code

Call	Result
Top 3 debits	T042 1432.93 · T057 962.80 · T050 903.91
Failed payments	`[P003, P012]`; source still 12
`anyMatch` elements inspected	3 (stops at the first match)
First powers of 2 over 1000	`[1024, 2048, 4096]`

Call	Result
Weekly review dates (3-arg iterate)	2026-11-02 … 2026-11-30 (5)
Total seats over 8 flights	2078
`HashSet` ORDERED?	false; `List`: true
`parallel().sequential()`	`isParallel() == false`

Anti-patterns → fixes

Don't	Do
call two terminals on one stream	keep the list / `Supplier<Stream>`
`limit(3).sorted(...)` for "top 3"	`sorted(...).limit(3)`
`iterate(seed, f)` with no limit	`limit`, `takeWhile` or 3-arg `iterate`

Don't	Do
peek for business logic	peek only to debug
`Files.lines(p)` left open	`try (var l = Files.lines(p))`
`forEach(list::add)` into an outer list	`toList()` / `collect`

Lesson 06 · sticky notes

Cut out, stick on your wall, and read one every morning.

No terminal, no work

Intermediate ops only describe.
The terminal op says GO.

One element at a time

filter P001, map P001,
then P002… (vertical order)

sorted is a barrier

It must see everything
before it emits the first element.

Streams are one-shot

Second terminal → IllegalStateException.
Use a `Supplier<Stream>`.

Infinite? Then limit it

`iterate(1, n -> n * 2)` never ends
without limit / takeWhile.

Java 9 · 16

9: iterate(3 args), takeWhile,
dropWhile, ofNullable
16: mapMulti, toList()

Order of ops matters

sorted → limit = top 3.
limit → sorted = first 3, sorted.

The source is untouched

A stream stores nothing.
Close `Files.lines` with try.